

CO2

Assessment

- 01

90/100

Name : A. Bhavya

Reg no : 192424417

Course code : CSA0613

Course Name : Design and Analysis of algorithm

Selection Sort - Dataset with Zero and Repetition

Apply the Brute force technique using selection sort on the dataset $[16, 0, 9, 16, 4, 0]$. Clearly interpret the problem and justify the use of selection sort for this dataset. Perform the sorting step-by-step and show the intermediate array after each pass. Count the no. of comparisons and swaps made during execution. Analyze the time complexity in best, average and worst cases. Verify the correctness of the final sorted output and interpret the effect of repeated zero values on the sorting process.

Problem understanding

Given dataset:

$A = [16, 0, 9, 16, 4, 0]$

The objective is to sort the given dataset in ascending order using the selection sort algorithm. The algorithm repeatedly selects the smallest element from the unsorted part of the array and places it in its correct position. We also need to count the total number of comparisons and swaps, analyze the time complexity, and verify the final sorted output.

Method selection

Algorithm used : Selection sort

Reason for selecting the algorithm:

- It is a simple brute-force sorting algorithm
- It repeatedly selects the min element from unsorted part.
- It is easy to understand and suitable for small datasets.

Solution process

Initial Array : [16, 0, 9, 16, 4, 0]

Pass 1

0	1	2	3	4	5
16	0	9	16	4	0

compare all elements from index 0 to 5

Minimum element = 0 (index 1)

swap 16 and 0

0	16	9	16	4	0
---	----	---	----	---	---

Comparisons = 5 ; swaps = 1

Pass 2

0	1	2	3	4	5
0	16	9	16	4	0

Min element = 0 (index 5)

swap 16 and 0

Comparisons = 4

swaps = 1

0	0	9	16	4	16
---	---	---	----	---	----

pass 3:

0	0	9	16	4	16
---	---	---	----	---	----

Min element = 4 (index 4)

swap '9' and '4'

0	0	4	16	9	16
---	---	---	----	---	----

Comparisons = 3 ; swaps = 1

pass 4:

0	0	4	16	9	16
---	---	---	----	---	----

Min element = 9 (index 4)

swap '16' and '9'

0	0	4	9	16	16
---	---	---	---	----	----

Comparisons = 2, swaps = 1

pass 5:

0	0	4	9	16	16
---	---	---	---	----	----

Minimum element is already in correct position

No swap is required.

Array after pass 5:

0	0	4	9	16	16
---	---	---	---	----	----

Comparisons = 1, swaps = 0

Table

Pass	Array after pass	Comparisons	Swaps
Initial	[16, 0, 9, 16, 4, 0]	-	-
1	[0, 16, 9, 16, 4, 0]	5	1
2	[0, 0, 9, 16, 4, 16]	4	1
3	[0, 0, 4, 16, 9, 16]	3	1
4	[0, 0, 4, 9, 16, 16]	2	1
5	[0, 0, 4, 9, 16, 16]	1	0

Accuracy of Calculation:-

Total Comparisons = $5 + 4 + 3 + 2 + 1 = 15$

Total Swaps = $1 + 1 + 1 + 1 = 4$

Final sorted Array : [0, 0, 4, 9, 16, 16]

Time Complexity Analysis:-

Case	Time Complexity
Best case ✓	$O(n^2)$
Average case	$O(n^2)$
Worst case	$O(n^2)$

Interpretation of Results

- The selection sort algorithm successfully sorted the given dataset in ascending order.
- the duplicate values (16 and 0) are correctly placed in the sorted array.
- A total of 15 comparisons and 14 swaps were performed.
- Selection Sort is easy to implement and suitable for small datasets, but it is not efficient for large datasets because its time complexity is $O(n^2)$.

